

UNIVERSITÀ DEGLI STUDI DI BARI ALDO MORO
Semantics in Intelligent Information Access (2025-26)

Fact Verification over Knowledge Graphs

Filippo di Gravina
Matricola: 840297

Contents

1	Introduction	2
2	Dataset Analysis and Limitations	2
2.1	FactKG	2
2.2	Observed Limitations	2
2.3	Open World vs. Closed World Assumption	3
3	Knowledge Graph Retrieval	3
3.1	Serialisation and Storage Migration	3
3.2	Graph Traversal Strategies	3
3.3	Semantic Text Features	4
4	Algorithm 1: SLM-Guided Fact Verification	4
4.1	Architecture	4
4.2	Prompt Engineering	5
4.3	Models Evaluated	5
4.4	Key Results and Model Comparison	5
4.5	Ensemble and Majority Voting	6
5	Algorithm 2: Semantic Graph Beam Search	6
5.1	Overview	6
5.2	Subgraph Loading and Anti-Hub Filtering	7
5.3	Statistical Training Module	7
5.4	Claim Type Inference	7
5.5	Cumulative Beam Score	8
5.6	Bidirectional Semantic Beam Search	8
5.7	Type-Specific Traversal Logic	9
5.8	Negation Detection	9
5.9	WN18RR Generalisation	9
6	Additional Experiments	10
7	Experimental Results	10
7.1	Algorithm 1	10
7.2	Algorithm 2	11
7.3	Energy and Runtime	12
8	Conclusions	12

Abstract

This report presents the design, implementation, and experimental evaluation of two systems for automated fact verification against Knowledge Graphs. The first system combines KG context retrieval with a Small Language Model guided by Chain-of-Thought prompting. The second one is a fully symbolic pipeline based on semantically guided beam search, learned relation-transition statistics, and linguistic heuristics. Both systems are evaluated on the FactKG benchmark, which uses DBpedia as its knowledge base. Algorithm 1 achieves 78–81% accuracy across models and CoT configurations, surpassing the existing SLM-based state of the art (approximately 67–68%). Algorithm 2 achieves State Of The Art 88.49% accuracy, outclassing the previous 86.8% SOTA accuracy, without any learned neural parameters, and transfers with zero structural modifications to the WN18RR benchmark achieving 83% accuracy, providing evidence against dataset-specific overfitting.

1 Introduction

Automated fact verification is the task of determining whether a natural language claim is supported or refuted by a structured knowledge base. It sits at the intersection of natural language understanding, graph reasoning, and symbolic semantics. As knowledge bases grow in scale, the need for efficient and interpretable fact-checking systems is growing.

This project approaches the problem from two perspectives. The neural approach uses a language model that reasons over retrieved graph context expressed in natural language. The symbolic approach replaces neural inference entirely with structured graph traversal and statistical scoring. The benchmark is FactKG, a dataset of claims annotated against DBpedia covering five reasoning categories: single-hop, multi-hop, existence, negation, and multi-claim (conjunction of claims).

2 Dataset Analysis and Limitations

2.1 FactKG

FactKG is a large scale fact verification dataset in which each claim is paired with a set of DBpedia triples derived from the annotated entity set and a binary label (Supported / Refuted). The dataset contains approximately 108 000 training, 3 400 development, and 9 000 test instances distributed across five reasoning types. The underlying knowledge base is DBpedia, a RDF knowledge graph extracted from Wikipedia.

2.2 Observed Limitations

Despite its scale, FactKG exhibits some problems that complicated model design:

1. **Unsupported triples.** Some supported claims cannot be confirmed by any path through the provided DBpedia subgraph, either because the relevant triples are absent or fragmented across disconnected components.
2. **Uncontrolled entity substitution.** Refuted examples are often constructed by replacing one entity with a random alternative (this can be seen in the metadata type `coll:substitution`). These substitutions did not always guarantee the correct labeling.

These observations motivated a design choice common to both algorithms. The choice made is to place emphasis on robust semantic matching over exact graph lookup to try to retrieve the needed triples without perfectly precise annotation.

2.3 Open World vs. Closed World Assumption

Under the Closed World Assumption (CWA), any fact not present in the KG is assumed false. Under the Open World Assumption (OWA), absence of evidence is not evidence of absence. For encyclopedic KGs like DBpedia, which are inherently incomplete, CWA leads to an unacceptably high false-negative rate, because missing triples would be systematically misclassified as Refuted. Both algorithms operate under a soft OWA, accepting a claim as true if a supporting path is found and defaulting to Refuted only when traversal fails and the structural context provides no corroborating signals.

3 Knowledge Graph Retrieval

Efficient and semantically coherent retrieval of KG evidence data is an important and difficult part of this task. The following strategies were explored.

3.1 Serialisation and Storage Migration

The DBpedia graph is originally distributed as a serialised Python adjacency dictionary (.pickle file). Loading it fully into RAM causes frequent out of memory failures. To address this, the graph was migrated to a SQLite relational database: the adjacency dictionary is iterated in chunks of 100 000 triples, each (subject, relation, object) triple is inserted into a table, and indexes on the subject column and object column are created to speed up the queries.

3.2 Graph Traversal Strategies

Multiple subgraph extraction policies have been tested:

- **Direct path lookup:** retrieval of triples whose subject or object exactly matches a claim entity. It is very prone to errors due to name mismatches and variations.
- **K-hop neighbourhood:** expansion to all nodes reachable within k hops from seed entities. Effective for multi-hop claims but exponentially expensive without pruning.
- **DFS/BFS with depth limit:** bounded graph traversal to limit combinatorial explosion. Directional BFS provided better recall than DFS for most reasoning types.
- **Bidirectional search:** simultaneous expansion from source and goal entities, meeting in the middle. This halves the effective search depth and substantially reduces the nodes explored.
- **Anti-hub filtering:** nodes exceeding a degree threshold are excluded from the subgraph. Hub nodes connect to hundreds of thousands of neighbours without providing discriminative signal. The threshold was set at 2 500 edges per node. This value has been chosen empirically based on the dev test accuracy.
- **Beam search:** path and edges are scored and a beam width limit is imposed via a parameter. This method permits to explore the best path first, independently of their length.

3.3 Semantic Text Features

Several NLP layers were applied on top of graph retrieval to improve semantic alignment between claim vocabulary and KG relation labels:

Keyword extraction: removes a curated set of stop words. High frequency words (articles, prepositions, auxiliary verbs and pronouns) are removed from the claim text. The residual content words are then matched against cleaned relation labels.

WordNet-based semantic expansion: used in the BM25 pipeline and the symbolic intermediate system. It expands each claim token with its synonyms, hyperonyms (up to depth 3), hyponyms (up to depth 3), and derivationally related forms. This helps to bridge the lexical gap between claim vocabulary and KG relation labels. Antonym detection is used as a negation signal, and path similarity scores provide a soft fallback for tokens with no direct lexical match.

POS tagging and proposition decomposition: for multi claim instances, NLTK’s POS tagger identifies coordinating conjunctions and relative pronouns separating independent clauses. Each clause is verified independently and the results are combined with conjunction logic. Anaphoric subjects (claims where a pronoun refers to the previous subject) are recovered by inspecting the preceding chunk.

Tilde notation handling: DBpedia encodes inverse relations with a tilde prefix. Both algorithms detect tilde prefixed relations and swap the subject-object order accordingly.

BM25+ scoring over flattened triple strings was used in an intermediate symbolic pipeline, combining term-frequency statistics with WordNet expanded query tokens to rank candidate evidence triples. BM25+ was preferred over vanilla TF-IDF for its better handling of rare terms and resistance to document-length bias.

Triple form vs. free-text paraphrasing: an early design decision was whether to feed SLMs structured (subject, relation, object) triples or paraphrased natural language sentences. Experiments showed that structured triples cause models to struggle with reading the relational semantics (e.g. Subject: Albert Einstein | Property: birthPlace | Object: Ulm), while natural language sentences (e.g. The birthPlace of Albert Einstein is Ulm) are parsed more reliably, especially by models with limited context windows.

4 Algorithm 1: SLM-Guided Fact Verification

4.1 Architecture

Algorithm 1 follows a retrieval augmented generation paradigm. For each test instance, the extracted KG context is paraphrased into natural language and prepended to the claim. A Small Language Model then produces a binary verdict (True / False), preceded by a configurable number of Chain Of Thought reasoning sentences.

The pipeline is:

- (1) tilde-normalization and paraphrasing of KG triples into sentences of the form “The *relation* of *subject* is *object*”;
- (2) prompt construction with a system instruction, context block, and claim;
- (3) SLM inference with temperature 0.0 (to ensure a deterministic output);

(4) regex-based extraction of the True/False verdicts from the generated text.

4.2 Prompt Engineering

Three prompt strategies were evaluated:

Zero-shot: the model receives context and claim with a direct instruction. This is the baseline.

Few-shot: a small number of worked (context, claim, reasoning, verdict) examples are prepended. The improvement over zero-shot is consistent with the literature, but at the cost of longer prompts and higher latency.

Chain-of-Thought (CoT): the model is instructed to generate one, two, or three sentences of reasoning before the verdict. CoT prompting was explored with varying levels of specificity: some configurations asked the model to enumerate the graph edges it found relevant, while others requested a free form logical deduction. CoT consistently improved accuracy, particularly on negation and multi-hop instances where explicit reasoning steps are beneficial. However, unconstrained CoT on models below 3B parameters often produces circular reasoning that circles back to an incorrect conclusion.

All runs used temperature 0.0 throughout to ensure deterministic and reproducible output.

4.3 Models Evaluated

Almost twenty model variants were evaluated across Kaggle GPU kernels (P100/T4), Google Colab, and a local workstation. All models were loaded in their native FP16 precision unless otherwise noted. The evaluated models include:

- **Qwen3.5:** 0.8B, 4B, 9B (Q8_0).
- **Gemma 4:** E2B (FP16), E4B (Q8_0), 12B (Q4_M), 26B-A4B (IQ3_XXS).
- **SmolLM2:** 135M, 360M.
- **Granite 4.1:** 3B, 8B (Q8_0).
- **Phi-4 Mini 3.8B.**
- **LLaMA 3.2:** 1B, 3B.
- **LFM-2.5 8B-A1B**

Models below 1B (SmolLM2) exhibited near random accuracy and frequent parsing failures. Models in the 1B-4B range showed high sensitivity to prompt formulation, with accuracy swinging up to 10 percentage points across prompt variants.

4.4 Key Results and Model Comparison

Average accuracy across CoT configurations (varying the number of reasoning lines, the level of detail requested, and whether explicit graph edges were enumerated) is shown in Figure 1.

Granite 4.1 3B averaged 78% accuracy. The applied instruction-following fine tuning methodology produces a model that is good for structured reasoning prompts, despite having only 3B parameters.

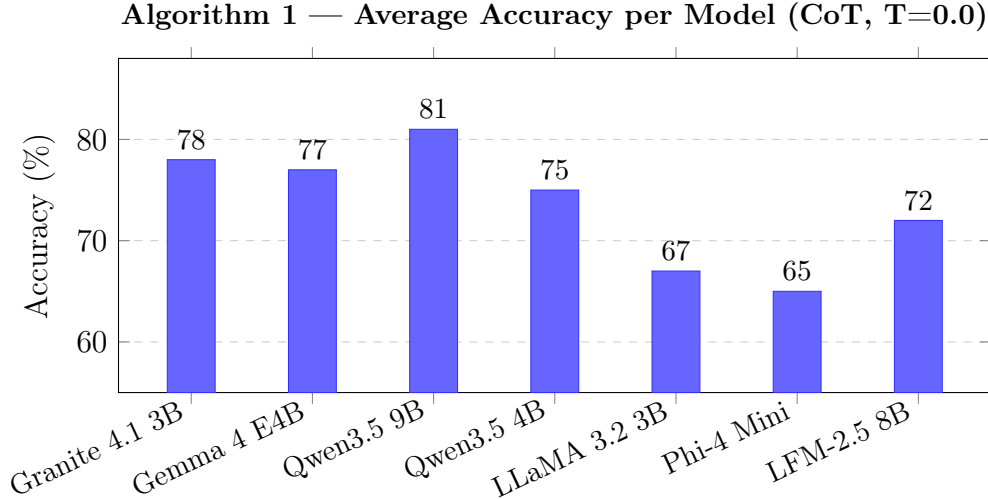


Figure 1: Average accuracy across CoT prompt configurations for the main models

Qwen3.5 9B at Q8_0 was the best-performing model at 81%. The hypothesis is that the new attention mechanism of the 3.5 series (Group Query Attention with extended RoPE), enables reliable handling of the contexts containing multiple KG triples. Q8_0 preserves fidelity far better than Q4 quantisations, which explains why the 9B at Q8 consistently outperformed even larger models at lower quantizations.

Without KG retrieval: all models tested on claims using only internal parametric knowledge (i.e. no context block provided) performed only marginally above random (around 52–56%). This demonstrates that even well trained SLMs do not have sufficient DBpedia coverage to verify fine grained factual claims without external retrieval and confirms that the accuracy gains of Algorithm 1 are genuine retrieval augmented improvements.

4.5 Ensemble and Majority Voting

Running different model with the same CoT prompt formulation and taking majority voting over the resulting predictions consistently reduced variance and improved accuracy by 1–2 percentage points, confirming that SLM decisions on borderline cases exhibit high entropy that can be partially cancelled by ensembling.

5 Algorithm 2: Semantic Graph Beam Search

5.1 Overview

Algorithm 2 is a fully symbolic retrieval fact verification system. It loads a targeted subgraph of DBpedia into memory, trains lightweight statistical structures on the FactKG training set, and classifies each test claim through a pipeline of type inference, keyword extraction, and semantically guided beam search. The system processes 9041 test instances in a local setting in under 2.5 minutes at approximately 70 claims/sec on a standard CPU, with no GPU requirement, outspeeding the average 0.5 claims/sec time of Algorithm 1.

5.2 Subgraph Loading and Anti-Hub Filtering

Rather than loading the full DBpedia graph, the system extracts a targeted subgraph anchored on the entities appearing in the claims. Entity names are normalised to their canonical DBpedia form (spaces replaced by underscores, Unicode normalised via NFKD decomposition, quotes stripped), and all aliases are expanded. The database is queried in chunks of 250 entities to avoid RAM spikes.

The degree threshold of 2 500 is applied and hub nodes exceeding this value are excluded. The threshold is chosen based on the accuracy on the dev test. Nodes below this threshold cover the vast majority of informative entities.

After filtering, the subgraph is stored in RAM as a dictionary of adjacency lists. Each edge stores (neighbour_id, cleaned_relation, raw_relation, direction), where direction is F (forward) or B (backward). This allows the beam search to treat edge direction as an explicit feature.

5.3 Statistical Training Module

The statistical trainer learns from the training set without neural networks:

Token polarity scores. For each content token appearing in training claims, the system records the fraction of times it appears in Supported ones. Tokens with polarity below 0.20 are treated as negation-correlated. The threshold 0.20 was chosen because tokens that appear in true claims less than 20% of the time appeared to be counter-factual indicators regardless of their syntactic role.

AMIE-style transition probabilities. For each pair of consecutive relations (r_1, r_2) in training evidence paths, the conditional probability $P(r_2 | r_1)$ is estimated as:

$$P(r_2 | r_1) = \frac{\text{count}(r_1, r_2)}{\text{count}(r_1)} \quad (1)$$

This is analogous to the rule-body-to-head confidence used in AMIE rule mining, adapted here to a sequential path context. Transitions are learned separately in the forward and backward directions to account for asymmetric relation semantics.

Token-to-relation alignment. For each claim token, the system counts its co-occurrence frequency with each KG relation label in true training instances. This produces a learned soft alignment that supplements lexical keyword matching: if a token has historically co-occurred with a particular relation, that relation receives a scoring bonus even without exact string match.

Direction inversion scores. For each claim token, the system tracks whether the entity order in the claim text is inverted relative to the graph edge direction in true training instances. This allows detection of passive and reversed constructions (e.g. “Poland is where Marie Curie was born” vs. “Marie Curie was born in Poland”).

5.4 Claim Type Inference

Before traversal, the claim is classified to a type specific handler.

1. If the entity set contains an atomic proposition, the claim is classified as `num1` (single-hop: find the entity, verify the attribute value).

2. If the entity set contains exactly one non-literal entity, the claim is classified as **existence** (does this entity appear with context consistent with the claim?).
3. If the claim contains conjunction markers and three or more entities, it is classified as **multi claim**.
4. Otherwise, a probabilistic fallback ranks possible types using the token-type distribution learned during training.

This routing is motivated by the observation that a generic multi-hop traversal is inefficient and often misleading for single-hop or existence queries, where the answer can be obtained with targeted 1–2 hop lookups.

5.5 Cumulative Beam Score

At each step of the beam search, the cumulative score for reaching node n via relation r from parent p is:

$$\text{score}(n) = \text{score}(p) + \underbrace{\log(m_f + \delta_f)}_{\text{keyword match}} + \underbrace{\log(1 + P_{\text{AMIE}} \cdot \lambda)}_{\text{transition bonus}} - \underbrace{\alpha \cdot \ell}_{\text{length penalty}} \quad (2)$$

where:

- $m_f \in [0, 1]$ is the fraction of claim keywords present in the cleaned relation label.
- $\delta_f = 0.20\text{--}0.25$: a floor offset that prevents $\log(0)$ when no keyword matches and provides a small base score for unmatched edges. The value keeps unmatched edges competitive enough not to be pruned immediately, while penalising them with respect to matched ones. The logarithm is used rather than a raw score because it provides a natural additive formulation of probabilities and compresses the dynamic range.
- P_{AMIE} is the learned transition probability from the previous relation to the current one.
- $\lambda = 4.5\text{--}6.0$: amplifies the AMIE bonus. Chosen empirically so that a well-supported transition (probability ≈ 0.1) adds roughly the same contribution as a moderate keyword match.
- $\alpha = 0.04\text{--}0.05$: length penalty per hop, discouraging unnecessarily long paths that are more likely to be coincidental than meaningful.
- ℓ : current path length.

The beam width of 150 (50 in the WN18RR variant) controls the breadth accuracy tradeoff. A width of 150 was found to be the practical ceiling beyond which runtime increases without further accuracy gain on this dataset.

5.6 Bidirectional Semantic Beam Search

For multi-hop claims, a bidirectional beam search expands simultaneously from the source entity and from the goal entity. When the two frontiers share a common intermediate node, a semantic compatibility check verifies that:

- (1) the two meeting relations have non-negligible AMIE transition probability, or
- (2) the intermediate node’s label contains at least one claim keyword. This two-condition gate prevents hub nodes from acting as spurious meeting points.

The max depth is 4 hops for FactKG and 3 hops for WN18RR (consistent with the shorter paths in lexicosemantic ontologies).

5.7 Type-Specific Traversal Logic

Single-hop (num1): the literal value is extracted from the entity set and matched against node labels in the 1-hop and 2-hop neighbourhood of the nominal entity. A confidence score combines lexical keyword overlap with the learned token to relation alignment. A path is accepted if the score exceeds 0.85, a threshold calibrated so that a relation label with a moderate keyword match and a reasonable training data alignment passes, while relations with no connection to the claim vocabulary do not. The 2-hop extension handles structural intermediaries.

Existence: one nominal entity is present. The system checks whether the entity appears in the subgraph with adjacent relations that are semantically consistent with the claim. The score threshold is set to 1.2, slightly higher than the single hop case to compensate for the absence of a target value to pin the path.

Multi-claim: entities are sorted decreasingly by degree and independent sub claims connecting the hub to each leaf entity are verified separately. The overall claim is accepted if at least 75% of sub-claims are confirmed. The 75% soft majority accounts for the fact that some sub claims may touch missing KG triples while the dominant evidence still supports the claim.

Multi-hop: bidirectional beam search up to 5 hops. Both a forward only AMIE guided search and the bidirectional search are attempted, and the claim is accepted if either succeeds. The forward-only fallback catches cases where the goal entity is not found in the subgraph but an intermediate confirmation node is.

5.8 Negation Detection

Negation is treated as a composite signal from three independent sources:

1. Explicit negation words in the claim (e.g. “not”, “never”).
2. Token polarity: if more than 50% of the claim’s content tokens have training polarity below 0.20 (i.e. appear predominantly in refuted claims), the claim is flagged as negated.
3. Structural type: if type inference assigned the negation category.

When negation is detected, the assigned truth value of the claim is inverted.

5.9 WN18RR Generalisation

WN18RR is a knowledge graph completion benchmark derived from WordNet 18, a lexicosemantic ontology structurally very different from DBpedia: entities are WordNet synsets rather than named entities, and relations are lexicosemantic (hypernymy, meronymy, derivation, similarity). The two datasets differ in their schema, vocabulary, triple format, and task formulation (KG completion with generated negatives vs. claim verification with human written natural language claims).

Algorithm 2 was ported to WN18RR by:

- (i) parsing the tab separated triple format

- (ii) applying human readable relation templates (e.g. hypernym $\rightarrow$ “{s} is a type of {o}”) to generate claim-like strings for keyword extraction
- (iii) re-computing AMIE transitions from the WN18RR training split
- (iv) generating negatives via random entity substitution, balanced 50/50 with positive triples.

The same system achieves 83% accuracy with zero structural modifications on the WN18RR test set (6 068 instances), strongly suggesting that the 88.49% result on FactKG reflects a genuine algorithmic capability rather than a dataset specific pattern learnt through implicit bias. WN18RR was chosen specifically as a second benchmark because it tests the algorithm on a fundamentally different KG schema and vocabulary domain.

6 Additional Experiments

Classical ML (LogReg, SVM, cross-validation). TF-IDF features extracted from claim text and paraphrased KG triples fed into logistic regression and SVM classifiers yielded 52–56% accuracy under cross-validation, confirming that simple embeddings and form features alone are insufficient for structured reasoning.

Sentence embeddings (DeBERTa, MiniLM, DistilBERT). Pre trained sentence encoders were used to compute similarity between claim embeddings and triple embeddings. The results are consistent with what the dataset authors reported.

Fine-tuning with LoRA/QLoRA. Parameter efficient fine tuning of smaller models on the FactKG training set was explored. Resource constraints (30 hour GPU weekly quotas on Kaggle) prevented this approach. Only initial partial results were obtained. Results looked promising, but the heavy amount of resources needed did not allow to end this experiment.

Boolean logic model. A rule based system treating KG facts as atomic propositions and claim sentences as logical formulas was implemented. While interpretable, it achieved lower accuracy than the beam search approach due to the high amount of language subtleties.

Custom neural network from scratch. A small feedforward neural network was trained on vocabulary indexed (subject, relation, object) triples to test whether a minimal supervised model could learn triple truth from DBpedia structure alone. Results were not promising, as the mismatch between the training distribution (randomly sampled DB triples) and the test distribution (human written FactKG claims) was too large to bridge without structured retrieval.

7 Experimental Results

7.1 Algorithm 1

Figure 1 (Section 4) showed per model averages. Across all CoT configurations, Qwen3.5 9B Q8_0 was the most accurate at 81%, followed by Granite 4.1 3B at 78% and Gemma 4 E4B at 77%. The baseline literature reports approximately 67–68% for comparable SLMs. Algorithm 1 surpasses this by 10–14 percentage points.

7.2 Algorithm 2

Reasoning Type	Accuracy	FP	FN
Single-hop (num1)	87.75%	193	98
Existence	88.84%	87	58
Multi-hop	84.61%	93	226
Negation	81.66%	130	111
Multi-claim	91.31%	174	112
Global	88.49%	547	494

Table 1: Algorithm 2 results by reasoning type on the FactKG test set (9 041 instances).

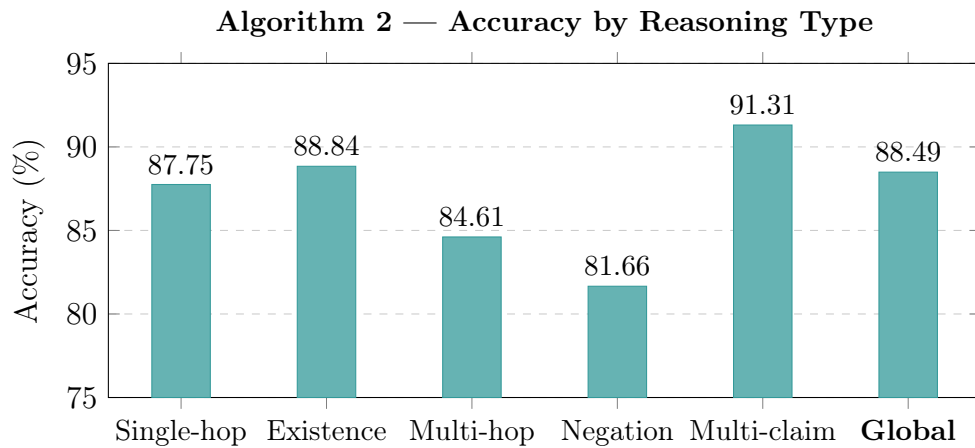


Figure 2: Per type and global accuracy of Algorithm 2 on the FactKG test set.

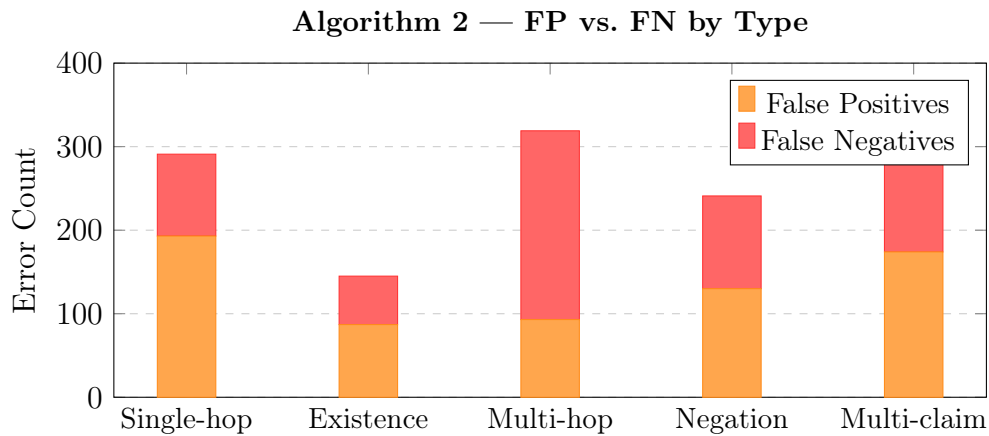


Figure 3: False positive and false negative breakdown per reasoning type.

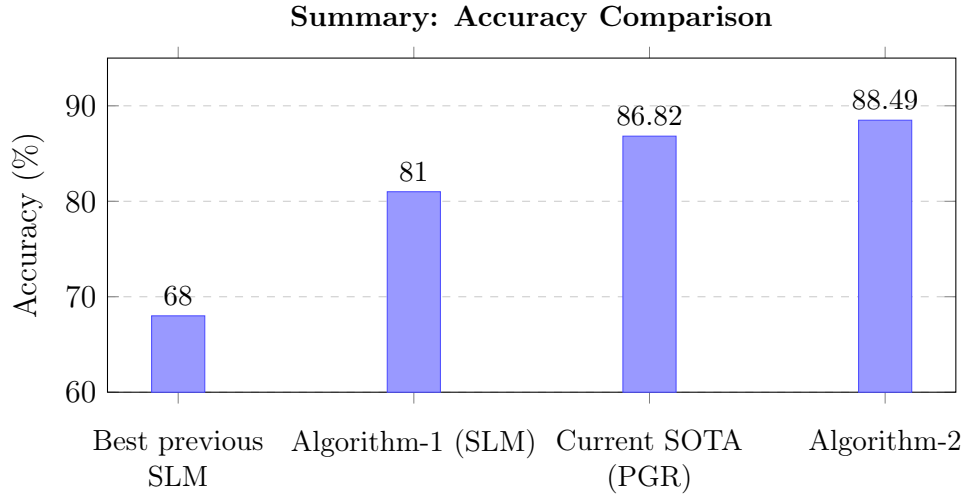


Figure 4: Summary accuracy comparison across systems.

7.3 Energy and Runtime

Algorithm 1 requires several GPU hours to process the full test set, depending on model size and the hardware available. Experiments were made on T4 GPUs available on Google Colab and Kaggle notebooks. Algorithm 2 processes the same 9 041 instances in approximately 2.5 minutes on CPU alone. The energy footprint difference is substantial and is a meaningful practical argument in favour of the symbolic approach for larger datasets.

8 Conclusions

This project developed and evaluated two complementary fact verification systems. Algorithm 1, combining KG retrieved context with a Small Language Model and Chain of Thought prompting, achieves 78–81% accuracy depending on the model, surpassing the SLM based state of the art by 10–14 percentage points. Algorithm 2, a fully symbolic semantically guided beam search with AMIE-derived transition statistics, achieves 88.49% global accuracy on FactKG and 83% on WN18RR. This additional result is the strongest evidence that Algorithm 2’s performance may reflect genuine reasoning capability, because the two datasets differ fundamentally in schema, vocabulary, entity type, and task formulation.

Future directions may include an ensemble combination between Algorithm 2 as a fast first pass, and LoRA fine tuned small language models as the main semantic evaluator.